Plataforma de autoaprendizaje digital

P1, G2

Arquitectura con modelo C4

Entrega 1 · Entregable E5 · docs/arquitectura.pdf

Equipo Backend

JOSE DAVID MARTINEZ FERNANDEZ

GABRIEL OMAR PAEZ ROLONG

ALFREDO DE JESUS TOUS VILLAREAL

13 de noviembre de 2026

Versión 1

Contenido

1. Introducción 3
2. Nivel 1 — Contexto 4
3. Nivel 2 — Contenedores 6
4. Nivel 3 — Componentes de la API 9
5. Estructura de carpetas del backend 12
6. Decisiones técnicas justificadas 14
7. Fragmento de código: módulo que une el caso de uso con el repositorio 15

Figuras

Figura 1. Diagrama de contexto (C4, nivel 1) 5
Figura 2. Diagrama de contenedores (C4, nivel 2) 8
Figura 3. Diagrama de componentes de la API (C4, nivel 3) 11

1. Introducción

Este documento describe la arquitectura de la **Plataforma de autoaprendizaje digital** del CLENA (proyecto P1, Guías CLENA): una aplicación web con guías paso a paso para que los visitantes de la Sala de Internet, en su mayoría adultos mayores, aprendan por su cuenta tareas digitales básicas.

La arquitectura se documenta con el **modelo C4**, que explica un sistema con diagramas de distinto nivel de detalle, como si se hiciera *zoom*. Este documento incluye los tres primeros niveles:

- **Nivel 1, Contexto:** el sistema como una sola caja, las personas que lo usan y los sistemas externos con los que se relaciona.
- **Nivel 2, Contenedores:** las piezas que se despliegan por separado (aplicación web, API, base de datos) y el protocolo con el que se comunican.
- **Nivel 3, Componentes de la API:** la API, organizada en las cuatro capas de Clean Architecture.

Todos los diagramas se construyeron a partir del código que está en la rama `main` del repositorio: los nombres de clases, carpetas, tecnologías y puertos que aparecen son los reales. Cada figura incluye su leyenda, la tecnología de cada elemento y una etiqueta en cada relación que indica qué se comunica y por qué protocolo.

Después de los diagramas, el documento presenta la estructura de carpetas del backend (sección 5), las decisiones técnicas con su justificación (sección 6) y el fragmento de código del módulo de NestJS que une un caso de uso con la implementación del repositorio (sección 7).

2. Nivel 1 — Contexto

La Figura 1 muestra la plataforma como una sola caja y las tres clases de persona que la usan. Los tres actores corresponden a los tres roles que existen en el código (`usuario`, `personal_clena` y `administrador`) todos entran por el navegador web de los computadores de la sala o de su propio equipo.

Tabla 1. Actores del sistema

Actor	Rol en el código	Qué hace en la plataforma
Usuario público	`usuario` (o sin sesión)	Consulta el catálogo de guías por categoría, busca guías por palabra clave y sigue una guía paso a paso sin necesidad de iniciar sesión. Si crea una cuenta (aceptando el aviso de privacidad) puede guardar guías como favoritas. *Planeado:* decir si una guía le sirvió y enviar dudas que sirvan para crear nuevas guías.
Personal del CLENA	`personal_clena`	Monitor o asistente de la sala. Crea, edita, publica y elimina guías desde la pantalla de gestión, sube las imágenes de los pasos y actualiza el horario y las normas de la sala. *Planeado:* revisar las dudas de los usuarios para decidir qué guías crear.
Administrador	`administrador`	Responsable de la plataforma. Tiene todos los permisos: además de lo anterior, administra las categorías del catálogo, los usuarios, los roles y los permisos de cada rol.

La tabla describe los permisos con los que se siembra cada rol (`users/domain/constants/default-roles.ts`). La API no autoriza por nombre de rol sino por permiso (`guides:manage`, `categories:manage`, `favorites:manage`…), y el administrador puede ajustar los permisos de los roles inferiores. Lo marcado como *planeado* aún no está implementado, pero los permisos ya existen en el código (`help-requests:create` para el usuario y `help-requests:read` para el personal); la opinión sobre la guía es la historia HU-11.

2.1 Sistemas externos

En esta entrega la plataforma **no depende de ningún sistema externo**: la autenticación es propia (JWT emitidos por la API) y las imágenes se guardan dentro del mismo sistema. Por eso el diagrama no dibuja cajas de sistemas externos y lo declara de forma explícita.

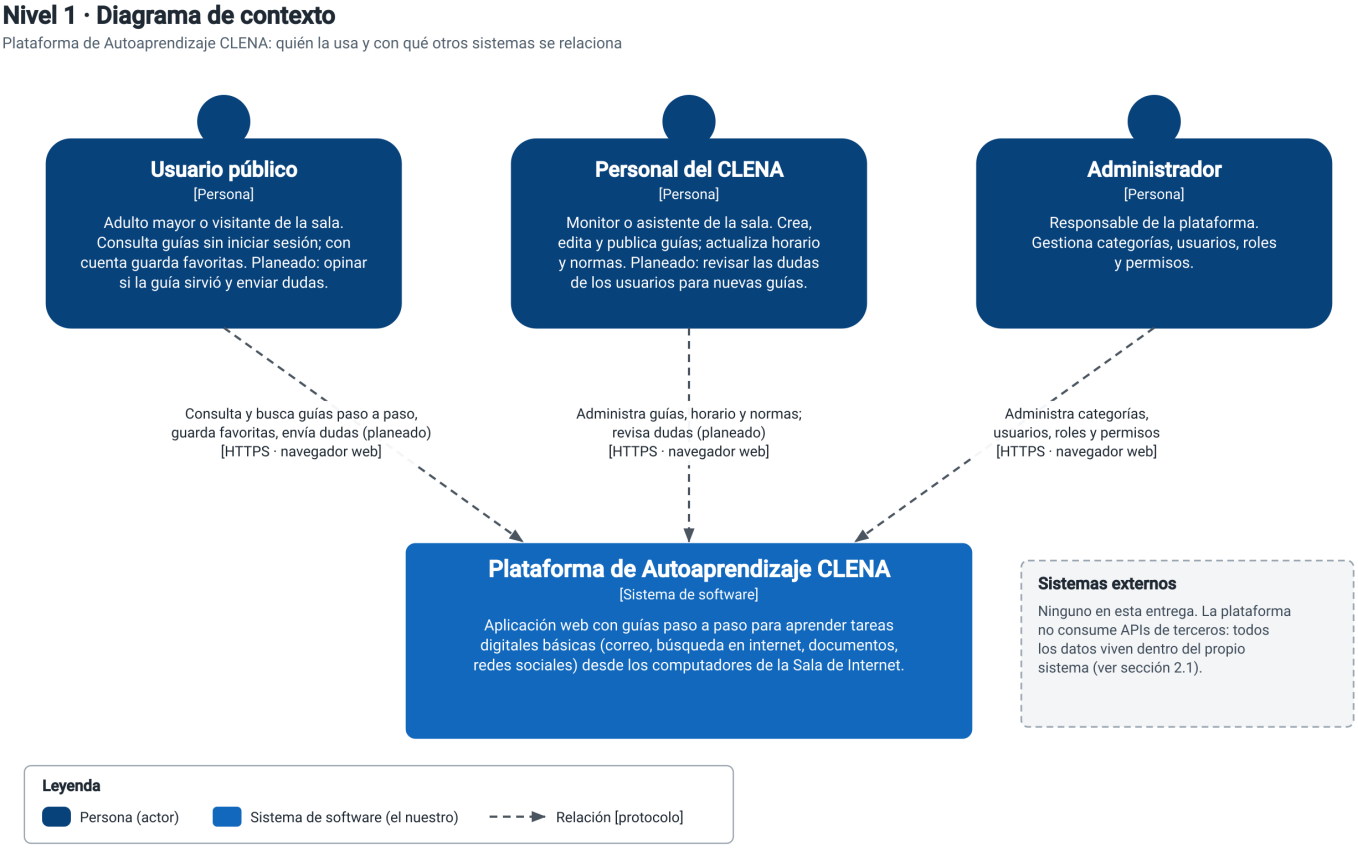


Figura 1. Diagrama de contexto (C4, nivel 1) de la Plataforma de Autoaprendizaje CLENA: actores y relaciones con el sistema.

3. Nivel 2 — Contenedores

La Figura 2 abre la caja del sistema y muestra sus contenedores, es decir, las piezas que se ejecutan por separado. Coinciden con los tres servicios del archivo `docker-compose.yml` (`frontend`, `backend` y `db`), más la aplicación React, que se descarga del servidor web y se ejecuta en el navegador de la persona.

Tabla 2. Contenedores y su tecnología

Contenedor	Tecnología	Responsabilidad
Servidor web	Nginx (imagen `nginx:alpine`), publicado en el puerto 8080	Entrega al navegador los archivos estáticos de la aplicación (HTML, JavaScript y CSS generados por Vite). No tiene lógica de negocio.
Aplicación web (SPA)	React 19, TypeScript, Vite, React Router, TanStack Query, Axios	Interfaz accesible que se ejecuta en el navegador: catálogo, búsqueda, guía paso a paso, favoritos, inicio de sesión, registro y pantalla de gestión para el personal.
API REST	NestJS 12, TypeScript, Node.js 26, puerto 3000	Concentra las reglas de negocio con Clean Architecture. Expone los recursos bajo `/api/v1`, la documentación Swagger en `/docs` y el chequeo `/api/v1/health`. Autentica con JWT y autoriza por permisos.
Base de datos	PostgreSQL 18 (imagen `postgres:18-alpine`), puerto 5432	Guarda usuarios, roles y permisos, sesiones (*refresh tokens*), categorías, guías con sus pasos, favoritos, horario y normas de la sala, y las imágenes de las guías.

Tabla 3. Conexiones entre contenedores

Conexión	Protocolo y formato	Qué se comunica
Persona → Aplicación web	HTTPS, desde el navegador	La persona navega por las pantallas de la aplicación.
Servidor web → Aplicación web	HTTPS; HTML, JS y CSS	El navegador descarga la aplicación una sola vez; a partir de ahí la SPA habla con la API.
Aplicación web → API REST	HTTPS con JSON (REST); cabecera `Authorization: Bearer <JWT>`; `multipart/form-data` para subir imágenes	Consultas y operaciones sobre guías, categorías, favoritos, sala, usuarios y sesión. Las imágenes de los pasos se descargan con `GET /api/v1/guides/images/:id`.
API REST → Base de datos	SQL sobre TCP (puerto 5432), mediante TypeORM y el driver `pg`	Lectura y escritura de todas las tablas; las migraciones versionadas se ejecutan al arrancar el contenedor.

> En el entorno local de `docker compose` los contenedores se publican por HTTP (`localhost:8080` y `localhost:3000`). El diagrama muestra HTTPS porque es el protocolo previsto para el despliegue en la sala.

3.1 Almacenamiento de imágenes

Los únicos archivos de la plataforma son las imágenes de los pasos de las guías. Hoy se guarda el **archivo completo dentro de PostgreSQL**: la tabla `guide_images` tiene una columna `data` de tipo `bytea` con los bytes de la imagen, junto con su nombre, tipo (PNG, JPEG, WebP o GIF) y tamaño, con un máximo de 5 MB por imagen. La API las entrega con `GET /api/v1/guides/images/:id`.

Se eligió así por simplicidad: no hace falta otro contenedor ni un volumen de archivos, y una sola copia de seguridad de la base incluye las imágenes. **Esta decisión está en revisión.** Si el número de imágenes crece, la alternativa es guardar los archivos en disco (un volumen del servidor) y en la base solo la ruta, lo que aligera la base y permite que Nginx sirva las imágenes directamente. Como el acceso pasa por la interfaz `GuideImageRepository`, el cambio solo afectaría a la capa de infraestructura.

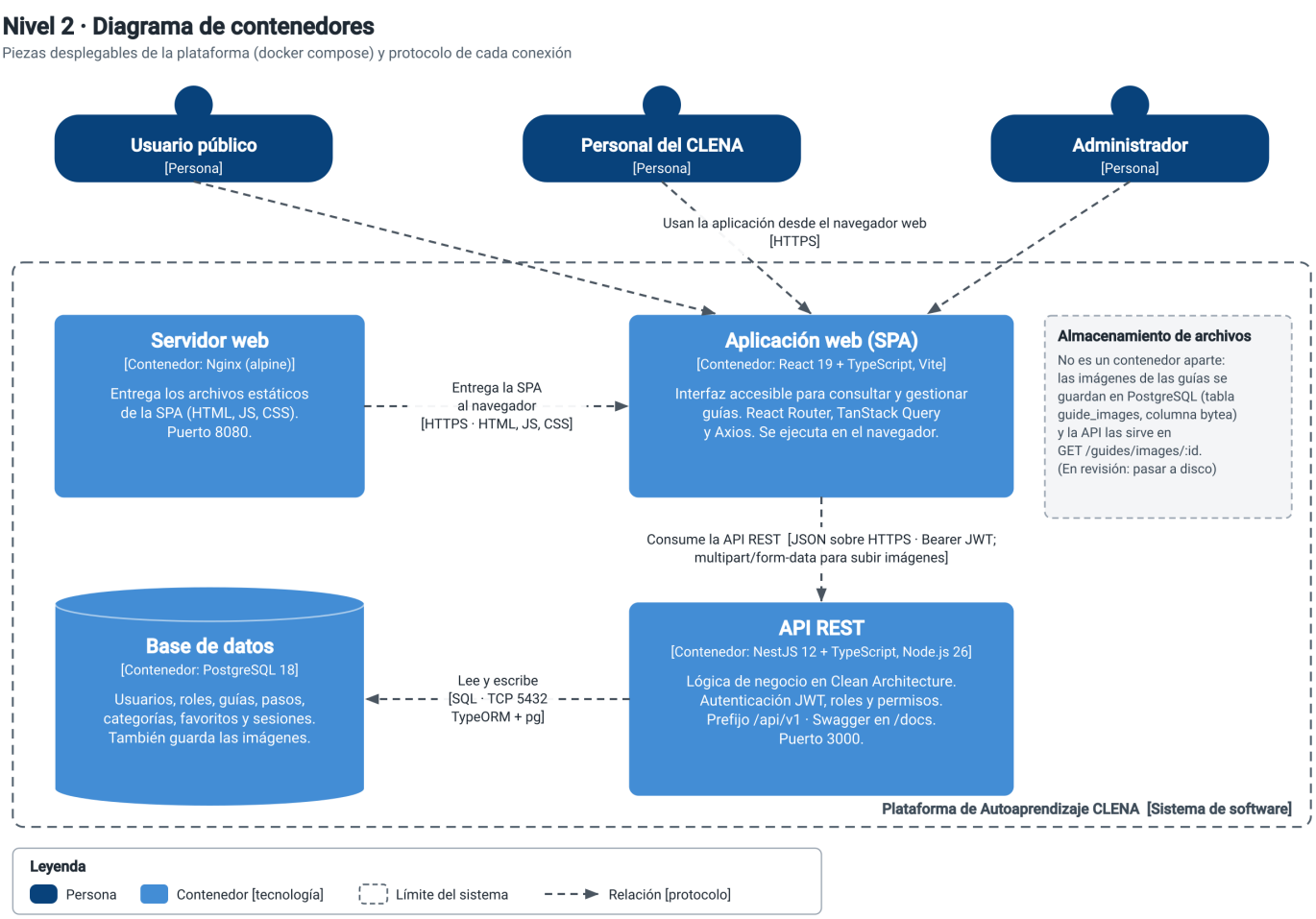


Figura 2. Diagrama de contenedores (C4, nivel 2): tecnología de cada contenedor y protocolo de cada conexión.

4. Nivel 3 — Componentes de la API

La Figura 3 abre el contenedor «API REST». El código está dividido en **módulos de negocio** (`auth`, `users`, `guides`, `categories`, `favorites`, `room-info` y `privacy`) y cada módulo repite las mismas **cuatro capas de Clean Architecture**. El diagrama agrupa los componentes por capa y lista las clases reales de cada grupo.

Tabla 4. Capas de la API y qué contiene cada una

Capa	Componentes	De quién depende
Presentación `presentation/`	Controladores REST, DTOs validados con `class-validator` y documentados con Swagger, *guards* globales (`JwtAuthGuard`, `PermissionsGuard`) y el `GlobalExceptionFilter`, que devuelve todos los errores con el mismo formato.	De Aplicación: el controlador solo traduce HTTP y llama a `execute()` del caso de uso.
Aplicación `application/`	Un caso de uso por acción del negocio (`LoginUseCase`, `CreateGuideUseCase`, `AddFavoriteUseCase`…), los puertos que necesita del exterior (`TokenService`, `PasswordHasher`, `CategoryLookup`…) y los errores de aplicación.	Solo del Dominio. De NestJS usa únicamente los decoradores de inyección (`@Injectable`, `@Inject`).
Dominio `domain/`	Entidades (`User`, `Guide`, `GuideStep`, `Category`, `Favorite`…), objetos de valor (`Email`, `SearchQuery`), enumeraciones (`Permission`, `GuideStatus`), errores de dominio e **interfaces de repositorio** con su token de inyección.	De nadie. Es TypeScript puro.
Infraestructura `infrastructure/`	Implementaciones de los repositorios con TypeORM, entidades del ORM y *mappers*, adaptadores de seguridad (`JwtTokenService`, `BcryptPasswordHasher`), semillas de datos y migraciones.	Del Dominio y de Aplicación: implementa sus interfaces y puertos.

4.1 La regla de dependencia

En el diagrama, una flecha significa «depende de» (es decir, «importa a»). Todas las flechas terminan apuntando hacia el Dominio:

- Presentación → Aplicación → Dominio: el controlador invoca el caso de uso y este trabaja con entidades e interfaces del dominio.
- Infraestructura → Dominio: los repositorios de TypeORM *extienden* las clases abstractas del dominio; el dominio nunca importa a TypeORM.

- Infraestructura → Aplicación: los adaptadores de seguridad implementan los puertos definidos por la capa de aplicación.

El caso de uso necesita guardar datos, pero no conoce la base de datos: pide «algo que cumpla la interfaz `FavoriteRepository`» y el módulo de NestJS le entrega la implementación de TypeORM en tiempo de ejecución (inversión de dependencias; se detalla en la sección 7).

> **Verificación en el código.** Se buscó en todas las carpetas `domain/` del backend cualquier `import` de `@nestjs/*`, `typeorm` o `class-validator` y no existe ninguno: el dominio no tiene dependencias de framework. Gracias a eso los casos de uso se prueban con repositorios en memoria (`test/support/fakes.ts`), sin levantar NestJS ni PostgreSQL.

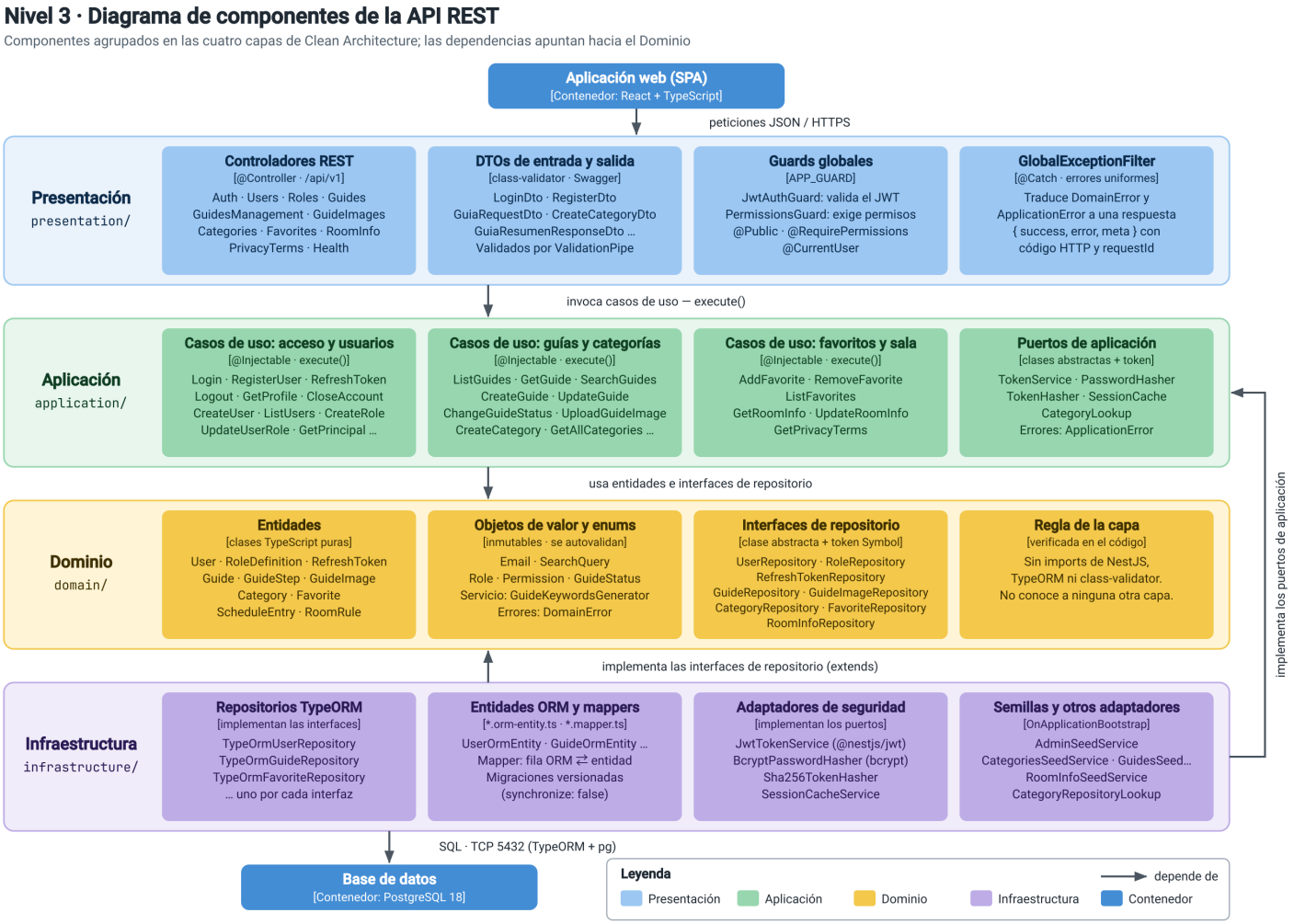


Figura 3. Diagrama de componentes de la API REST (C4, nivel 3), agrupados en las cuatro capas de Clean Architecture. Las dependencias apuntan hacia el Dominio.

5. Estructura de carpetas del backend

El Listado 1 muestra la estructura real de `backend/`. Hay una carpeta por módulo de negocio y, dentro de cada una, una carpeta por capa. Se despliega completo el módulo `favorites` como ejemplo; los demás módulos siguen el mismo esquema. Lo que usan varios módulos vive en `shared/`, también separado por capa.

```
backend/
├── Dockerfile                    imagen de producción; al iniciar ejecuta las migraciones
├── test/                         pruebas e2e (*.e2e-spec.ts) y dobles en memoria (support/fakes.ts)
└── src/
    ├── main.ts                   arranque: Swagger en /docs, CORS y puerto
    ├── app.setup.ts              prefijo /api/v1, ValidationPipe global y filtro global de errores
    ├── app.module.ts             módulo raíz: configuración, TypeORM y módulos de negocio
    ├── modules/
    │   ├── auth/                 registro, inicio de sesión, refresh token, cierre de cuenta
    │   ├── users/                usuarios, roles y permisos
    │   ├── guides/               guías, pasos, búsqueda e imágenes
    │   ├── categories/           categorías del catálogo
    │   ├── room-info/            horario y normas de la sala
    │   ├── privacy/              aviso de privacidad
    │   └── favorites/            guías favoritas (módulo de ejemplo, desplegado)
    │       ├── favorites.module.ts                       une tokens con implementaciones
    │       ├── domain/
    │       │   ├── entities/favorite.entity.ts          entidad de dominio
    │       │   └── repositories/favorite.repository.ts  interfaz + token
    │       ├── application/
    │       │   ├── use-cases/add-favorite.use-case.ts    caso de uso (+ .spec.ts al lado)
    │       │   ├── use-cases/remove-favorite.use-case.ts
    │       │   ├── use-cases/list-favorites.use-case.ts
    │       │   └── errors/favorite.errors.ts
    │       ├── infrastructure/persistence/
    │       │   ├── favorite.orm-entity.ts                tabla «favorites»
    │       │   ├── favorite.mapper.ts                    ORM ⇄ dominio
    │       │   └── typeorm-favorite.repository.ts        implementación del repositorio
    │       └── presentation/
    │           ├── controllers/favorites.controller.ts
    │           └── dtos/favorite-guide-param.dto.ts
    └── shared/
        ├── domain/errors/                DomainError
        ├── application/errors/           ApplicationError y errores comunes
        ├── application/models/           AuthenticatedUser
        ├── application/ports/            TokenService, PasswordHasher, TokenHasher, SessionCache
        ├── infrastructure/database/      data-source.ts, typeorm-options.ts
        │   └── migrations/               migraciones versionadas (<timestamp>-Nombre.ts)
        ├── infrastructure/security/      BcryptPasswordHasher, Sha256TokenHasher
        ├── presentation/decorators/      @Public, @CurrentUser, @RequirePermissions
        ├── presentation/dto/ y errors/   formato de error y ValidationException
        ├── presentation/filters/         GlobalExceptionFilter
        ├── presentation/guards/          JwtAuthGuard, PermissionsGuard
        └── presentation/health.controller.ts     GET /api/v1/health
```

Listado 1. Estructura de carpetas del backend (rama main).

Tabla 5. Dónde va cada tipo de archivo

Tipo de archivo	Ubicación dentro del módulo	Nombre
Entidad de dominio	`domain/entities/`	`*.entity.ts`
Objeto de valor	`domain/value-objects/`	`*.vo.ts`
Enumeraciones y constantes	`domain/enums/`, `domain/constants/`	`*.enum.ts`
Interfaz de repositorio y su token	`domain/repositories/`	`*.repository.ts`
Errores de negocio	`domain/errors/`	`*.errors.ts`
Caso de uso y su prueba unitaria	`application/use-cases/`	`*.use-case.ts`, `*.use-case.spec.ts`
Puerto hacia el exterior	`application/ports/` o `shared/application/ports/`	`*.port.ts`
Modelos de lectura y errores de aplicación	`application/models/`, `application/errors/`	`*.model.ts`, `*.errors.ts`
Entidad del ORM (tabla)	`infrastructure/persistence/`	`*.orm-entity.ts`
Mapper ORM ⇄ dominio	`infrastructure/persistence/`	`*.mapper.ts`
Implementación del repositorio	`infrastructure/persistence/`	`typeorm-*.repository.ts`
Datos semilla	`infrastructure/seeding/`	`*-seed.service.ts`
Controlador	`presentation/controllers/`	`*.controller.ts`
DTO de entrada o de respuesta	`presentation/dtos/`	`*.dto.ts`
Módulo de NestJS	raíz del módulo	`<modulo>.module.ts`
Migración	`shared/infrastructure/database/migrations/`	`<timestamp>-Nombre.ts`
Guards, filtros y decoradores	`shared/presentation/`	`*.guard.ts`, `*.filter.ts`, `*.decorator.ts`

TypeORM descubre las entidades por convención de nombre: la configuración carga `modules/**/infrastructure/**/*.orm-entity.ts`, de modo que una entidad del ORM colocada fuera de la capa de infraestructura no se registraría.

6. Decisiones técnicas justificadas

La Tabla 6 resume cada decisión, qué otra opción había y por qué se eligió.

Tabla 6. Decisiones técnicas

Decisión	Elección	Alternativas	Por qué
ORM	TypeORM	Prisma, Sequelize	Tiene integración oficial con NestJS y migraciones en TypeScript. Sus entidades se quedan en la capa de infraestructura, así el dominio no depende del ORM.
Validación	`class-validator` con `ValidationPipe` global	Zod, Joi	Es lo estándar en NestJS. La misma clase DTO valida la petición y documenta el endpoint en Swagger.
Logger	`Logger` integrado de NestJS	Pino, Winston	No agrega dependencias y escribe en consola, que Docker ya guarda. Alcanza para un solo servidor.
Componentes del frontend	Componentes propios con CSS; íconos `lucide-react`	Material UI, Chakra UI	Control total de tamaños, contraste y foco para adultos mayores, y una aplicación más liviana.

6.1 ORM: TypeORM

Cada tabla se define en un archivo `*.orm-entity.ts` dentro de infraestructura, y un *mapper* la convierte en entidad de dominio. Los cambios de la base se hacen con migraciones que se ejecutan solas al arrancar el contenedor, por lo que `docker compose up` deja la base lista. Prisma se descartó porque genera sus propios modelos y obliga a mantener otro esquema aparte.

6.2 Validación: class-validator

Cada DTO declara sus reglas con decoradores (`@IsEmail`, `@Length`...). El `ValidationPipe` global revisa toda petición antes de llegar al controlador y rechaza campos que el DTO no declara. Las reglas de negocio no van aquí: se validan en el dominio.

6.3 Logger: el de NestJS

Se usa la clase `Logger` de NestJS en el filtro global de errores y en las semillas. Los registros se ven con `docker compose logs backend`. Si más adelante hace falta algo más completo se puede cambiar a Pino sin tocar los casos de uso.

6.4 Componentes del frontend: propios

El público son adultos mayores, así que la interfaz usa letra grande (18 px), botones de mínimo 48 px y alto contraste. Son pocos componentes (botones, tarjetas, formularios, diálogo) y hacerlos con HTML y CSS es más simple que adaptar Material UI, cuyos estilos están pensados para otro público.

7. Fragmento de código: módulo que une el caso de uso con el repositorio

Ejemplo con el módulo de favoritos (`backend/src/modules/favorites/`).

El módulo. La línea
`{ provide: FAVORITE_REPOSITORY, useClass: TypeOrmFavoriteRepository }` le dice a NestJS: «cuando alguien pida `FAVORITE_REPOSITORY`, entrégale un `TypeOrmFavoriteRepository`».

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { GuidesModule } from '../guides/guides.module';
import { FAVORITE_REPOSITORY } from './domain/repositories/favorite.repository';
import { FavoriteOrmEntity } from './infrastructure/persistence/favorite.orm-entity';
import { TypeOrmFavoriteRepository } from './infrastructure/persistence/typeorm-favorite.repository';
import { FavoritesController } from './presentation/controllers/favorites.controller';
import { AddFavoriteUseCase } from './application/use-cases/add-favorite.use-case';
import { ListFavoritesUseCase } from './application/use-cases/list-favorites.use-case';
import { RemoveFavoriteUseCase } from './application/use-cases/remove-favorite.use-case';

@Module({
  imports: [TypeOrmModule.forFeature([FavoriteOrmEntity]), GuidesModule],
  controllers: [FavoritesController],
  providers: [
    { provide: FAVORITE_REPOSITORY, useClass: TypeOrmFavoriteRepository },
    AddFavoriteUseCase,
    RemoveFavoriteUseCase,
    ListFavoritesUseCase,
  ],
})
export class FavoritesModule {}
```

Listado 2. favorites.module.ts: el módulo enlaza el token con la implementación.

La interfaz y el token (dominio). Dice qué operaciones existen, sin decir cómo se hacen. El token es un `Symbol` que la identifica.

```
import type { Favorite } from '../entities/favorite.entity';

export abstract class FavoriteRepository {
  abstract exists(userId: string, guideId: string): Promise<boolean>;
  abstract add(favorite: Favorite): Promise<void>;
  abstract remove(userId: string, guideId: string): Promise<void>;
  /** Ids de guía favoritas del usuario, de la más reciente a la más antigua. */
  abstract listGuideIdsByUser(userId: string): Promise<string[]>;
}

export const FAVORITE_REPOSITORY = Symbol('FAVORITE_REPOSITORY');
```

Listado 3. domain/repositories/favorite.repository.ts: interfaz del repositorio y su token.

El caso de uso (aplicación). Pide el repositorio por su token y solo conoce la interfaz; no sabe que detrás está TypeORM.

```
@Injectable()
export class AddFavoriteUseCase {
  constructor(
    @Inject(FAVORITE_REPOSITORY) private readonly favorites: FavoriteRepository,
    @Inject(GUIDE_REPOSITORY) private readonly guides: GuideRepository,
  ) {}

  async execute(input: AddFavoriteInput): Promise<void> {
    const guide = await this.guides.findById(input.guideId);

    if (!guide || guide.status !== GuideStatus.PUBLISHED) {
      throw new FavoriteGuideNotFoundError();
    }

    const alreadyFavorite = await this.favorites.exists(input.userId, input.guideId);

    if (alreadyFavorite) {
      return;
    }

    await this.favorites.add(Favorite.create({ userId: input.userId, guideId: input.guideId }));
  }
}
```

Listado 4. application/use-cases/add-favorite.use-case.ts: el repositorio llega por token de inyección.

Si se cambiara TypeORM por otra tecnología, el caso de uso y la interfaz quedan igual: solo se escribe otra clase que extienda `FavoriteRepository` y se cambia una línea del módulo.